

CSE 751: Advanced Natural Language Processing

Final Examination: Questions and Answers

Examination preparation set

Contents

Part I: Topic-wise Questions and Answers	3
1 The Simple Recurrent Neural Network	3
2 Bidirectional RNNs and Recurrent Architectures	5
3 The Encoder–Decoder (Seq2Seq) Architecture	7
4 Attention over Encoder States	8
5 Self-Attention and Scaled Dot-Product Attention	9
6 Multi-Head Attention	10
7 Positional Encoding	11
8 Residual Connections, Add & Norm, and the Feed-Forward Layer	12
9 The Transformer Decoder: Masking and Cross-Attention	12
10 Tokenization: Byte-Pair Encoding and WordPiece	13
11 BERT	14
12 The GPT Family	16
13 Pretraining, Data and Scaling Laws	17
14 LLaMA	18
15 Long Context and Mixture of Experts	19
16 Quantization	21
17 PEFT, Pruning and Distillation	21
18 Instruction Tuning and Alignment	23
19 Prompting, In-Context Learning and Chain of Thought	25
20 Retrieval-Augmented Generation	27

21 Reasoning Models and the Agent Loop	29
22 Evaluation of LLMs	29
 Part II: Numerical and Worked Problems	 31
23 Attention Calculations	31
24 RNN Parameter Counting	32
25 Positional Encoding Values	32
26 Quantization Arithmetic	32
27 LoRA Parameter Count	33
28 KV Cache and Grouped-Query Attention	33
29 Mixture of Experts, Scaling and Context Cost	34
30 BERT Masking Arithmetic	34
 Part III: Mock Final Papers	 35
31 Mock Paper A	35
32 Mock Paper B	36
33 Mock Paper C	38

Part I: Topic-wise Questions and Answers

1 The Simple Recurrent Neural Network

Q1. Write a short note on the three weight matrices W , U and V of a simple RNN. (Not exceeding three sentences.)

Answer. W reads the current word, U reads the memory left over from the previous step, and V turns the memory into an answer, which gives $h^{(t)} = \tanh(Uh^{(t-1)} + Wx^{(t)})$ and $o^{(t)} = \text{softmax}(Vh^{(t)})$. They are kept separate because reading a new word, remembering the old ones, and producing an output are three different jobs. The same three matrices are reused at every word, which is why one small RNN can read a sentence of any length.

Q2. Write a short note on backpropagation through time. (Not exceeding three sentences.)

Answer. Backpropagation through time unrolls the loop into a chain, one copy per word, and then trains it like an ordinary deep network by sending the error backwards through every earlier word. Because the error is multiplied by a number at each step going back, the signal that reaches the first word is the product of many such numbers. The weights are updated only after the whole sentence, because the same three matrices helped cause the error at every position.

Q3. Why is the vanishing gradient problem harder to escape in an RNN than in a deep feed-forward network of the same depth? Explain in no more than three sentences.

Answer. A feed-forward network has a different weight matrix at each layer, so if one layer shrinks the signal too much, another layer can be learned larger to make up for it. An RNN uses the same matrix U at every step, so whatever shrinking happens once happens again and again, like multiplying 0.5 by itself fifty times. It cannot fix the shrinking at one step without breaking the forward computation at every other step, because there is only one matrix to change.

Q4. State the pros and cons of the simple RNN.

Answer.

Pros:

- It can read sentences of any length using the same small set of weights.
- It keeps a memory, the hidden state, that carries information forward from word to word.
- It uses very little memory: the state is one vector, no matter how long the sentence is.
- It can work on live input, handling each word the moment it arrives.
- The same cell handles many task shapes: one label per sentence, one label per word, or a whole new sentence.

Cons:

- It must read words one after another, so it cannot use a GPU properly.
- The error signal shrinks (or blows up) as it travels back through many words, so links between far-apart words are hard to learn.
- It only sees the words before the current one, never the words after.
- It forgets the same amount at every step, even when one particular word deserves to be kept much longer.

Q5. What problem does the simple RNN solve?

Answer.

- It handles input of *different lengths*. A normal network needs a fixed-size input; an RNN just runs its loop more times for a longer sentence.
- It makes *word order* matter. “dog bites man” and “man bites dog” end up with different memories.
- Each prediction can depend on *everything read so far*, not just the last few words.
- It does all this with a *fixed number of weights*, so a model trained on short sentences can still read long ones.
- Because the weights are shared, a pattern learned at word 3 also works at word 30.

Q6. What problems can a simple RNN not solve, no matter how well it is trained?

Answer.

- It cannot do the words in parallel. Step t needs the answer from step $t - 1$, so this is built into the design, not something training can fix.
- It cannot look at future words. Reading “book” it cannot check whether “a flight” comes next unless you run a second RNN backwards.
- It cannot give a word a short path back to a word 200 steps earlier. The information has to travel through every step in between.
- It cannot choose to keep one word longer than another, because the update rule is the same every time.
- One fine point the exam likes: the RNN *could* in theory hold a long-distance rule, but training will never find it. Being able to store something and being able to learn it are not the same thing.

Q7. What problems did the simple RNN give rise to?

Answer.

- **Vanishing gradients.** Training multiplies many small numbers together, so the first words get almost no learning signal.
- **Exploding gradients.** The same multiplication with numbers above one makes the loss jump to NaN, which is why cutting gradients down to a maximum size became standard.
- **Wasted hardware.** GPUs are built to do thousands of things at once, and a model that insists on one word at a time leaves most of that idle.
- **A need for right-hand context**, which led to running a second RNN backwards.
- **A need for memory you can control**, which led to gated recurrent versions, and eventually to attention replacing recurrence altogether.

Q8. When should a simple RNN be used, and when should it not be used?

Answer.

Use it when:

- The sequences are short and what matters is nearby, such as the last few letters of a word.
- Input arrives one item at a time and an answer is needed straight away, as in live typing prediction.
- Memory and battery are very tight, such as on a small device.
- You want a quick, simple baseline to compare a bigger model against.
- You are teaching or debugging, because you can watch what happens at each step.

Do not use it when:

- The answer depends on something said much earlier in the text.
- You have GPUs and want fast training, since the RNN will leave them idle.
- A word’s meaning depends on what comes after it, unless you add a backward pass.
- A pretrained Transformer is available, because it will be more accurate on almost every benchmark.

Q9. Counterfactual. Suppose W , U and V were *not* shared across time steps, and each position had its own copy. What would have happened?

Answer.

- The number of weights would grow with sentence length instead of staying fixed.
- The model could never read a sentence longer than the longest one it was built for, because there would be no weights for the extra positions.
- A rule learned at word 3 would not help at word 30. The model would have to learn the same rule all over again at every position.
- Much more training data would be needed, and the weights for late positions would almost never get practice, because long sentences are rare.
- The point to remember: sharing the weights is not a small saving. It is the whole reason an RNN works on sentences it has never seen.

Q10. Scenario. A hospital wants a model that reads a patient’s vital-sign readings as they arrive, one every second, and raises an alarm within one second of a dangerous pattern appearing. The patterns of interest span about ten readings. Which architecture should be used, and why?

Answer. Use a one-directional RNN. The pattern only spans about ten readings, so the shrinking-gradient problem never gets a chance to bite; the alarm must fire without waiting for future readings, which rules out a bidirectional model; and one RNN step is a single small matrix multiplication, which easily fits inside one second. A Transformer would have to look back over a longer and longer window at every tick, costing more for no gain at this short a range.

2 Bidirectional RNNs and Recurrent Architectures

Q11. Write a short note on the bidirectional RNN. (Not exceeding three sentences.)

Answer. A bidirectional RNN runs one RNN left to right and a second, separate RNN right to left over the same sentence, then joins the two memories at each word, usually by sticking them together end to end. Each word therefore ends up described by what came before it *and* what comes after it. The price is that the whole sentence has to arrive before any answer can be given.

Q12. What problem does the bidirectional RNN solve?

Answer.

- It fixes the “past only” blindness of a normal RNN.
- Many words are only settled by what follows. In “I will *book* a flight”, the word “flight” is what tells you “book” is a verb here.
- Every word gets described using the whole sentence, which is exactly what tagging jobs like part-of-speech tagging and name-finding need.

- It needs no change to the RNN itself. It just runs the same idea twice, in opposite directions.

Q13. What problems does a bidirectional RNN *not* solve, and what new problem does it create?

Answer.

Still not solved:

- Each direction is still word by word, so training is still slow on a GPU.
- The signal still has to travel through every step, so very long links are still hard to learn.
- It costs about twice the computing and twice the weights of a one-directional RNN.

New problem created:

- It kills live use. The backward pass cannot start until the last word arrives, so the model can no longer answer while the input is still coming in.

Q14. When should a bidirectional RNN not be used?

Answer.

- In live speech recognition, live captioning, or anything where the user expects output while still talking.
- In next-word prediction, because the words on the right contain the answer and the model would simply be cheating.
- When there is a strict time limit and the input is long, since you must wait for the whole thing first.
- When memory is tight, because you are now storing two sets of memories instead of one.

Q15. Scenario. A team builds a part-of-speech tagger with a one-directional RNN. Accuracy is fine overall but poor on ambiguous words. What would have happened if they had used a bidirectional RNN from the start?

Answer.

- The ambiguous words would have improved the most, because the word that settles the ambiguity usually comes *after* the ambiguous word.
- The clear words would barely change, since the left-hand context had already settled them.
- Training time and memory would roughly double.
- The tagger would stop working on half a sentence, so any live use of it would break.

Q16. Scenario. For sentiment classification, an engineer takes the *last* memory of an RNN as the summary of the sentence. Accuracy is good on short tweets but poor on long product reviews. What is going wrong, and what would be the solution?

Answer.

- **What is wrong.** The last memory is one fixed-size summary of everything. In a long review it has been rewritten dozens of times since the opening sentences, so those sentences have almost no influence left, and the decision is made mostly by the last few lines.
- **Fix 1.** Use *all* the memories, not just the last one: take their average, or take the largest value in each position. Now every part of the review has a say.
- **Fix 2.** Add an attention layer that learns *which* sentences to weight. This usually beats a plain average, because it can pick out the lines that actually carry the opinion.

- **Fix 3.** Replace the RNN with a pretrained Transformer encoder, where every word has a direct link to every other word. That removes the cause rather than working around it.

3 The Encoder–Decoder (Seq2Seq) Architecture

Q17. Write a short note on the RNN encoder–decoder. (Not exceeding three sentences.)

Answer. An encoder RNN reads the whole input sentence and squeezes it into its final memory, called the context vector, and a decoder RNN starts from that vector and writes the output one word at a time, feeding each word it writes back in as the next input. This splits reading from writing, so the output does not have to be the same length as the input. It is what made end-to-end machine translation possible without hand-built alignment tables.

Q18. What problem does the encoder–decoder architecture solve?

Answer.

- It handles jobs where input and output have *different lengths*, such as turning a 10-word English sentence into a 14-word Bangla one.
- It handles jobs where the output comes in a *different order*, since the decoder can write words in whatever order the target language wants.
- It removes the rule of one output per input, which an ordinary RNN tagger forces on you.
- One trained system replaces a whole pipeline of separate translation parts.

Q19. What problem did the basic encoder–decoder give rise to? Explain in no more than three sentences.

Answer. It creates a bottleneck: the whole input sentence, however long, has to fit into one fixed-size vector. So translation quality falls off sharply as sentences get longer, and the earliest words suffer most, because they have to survive the largest number of rewrites of that memory. Attention was invented specifically to remove this bottleneck.

Q20. What is teacher forcing? State one advantage and one drawback. (Not exceeding three sentences.)

Answer. During training the decoder is fed the *correct* previous word instead of the word it actually predicted. The advantage is fast, stable learning, because one early mistake does not wreck the rest of the sentence and every step starts from a clean input. The drawback is called exposure bias: when the model is actually used it must feed on its own imperfect output, which it never practised on, so mistakes pile up in long answers.

Q21. When should an encoder–decoder be chosen over a decoder-only model?

Answer.

- When there is a clear input document and a separate output to write: translation, summarising, fixing grammar.
- When the input should be read in *both* directions, which a decoder-only model cannot do because it is only allowed to look left.
- When the input is long and the output is short, since the encoder can read it once and the decoder can then refer back cheaply.
- When you want a smaller model built for one job rather than one huge general model.

Choose decoder-only instead when the task is open-ended writing, when one model must handle many tasks through prompts, or when you want the scaling behaviour that made GPT-style models dominant.

Q22. Counterfactual. A team trains an encoder–decoder translator *without* teacher forcing, feeding the decoder its own predictions from the very first epoch. What would have happened?

Answer.

- Early in training the predictions are close to random, so the model would spend its time learning how to continue nonsense.
- Learning would be far slower and might never settle, because the training signal is dominated by mistakes instead of by real language.
- Training could no longer be done for all output positions at once, since each step must wait for the previous prediction.
- One thing *would* get better: exposure bias. The model would be practising on the same kind of messy input it meets in real use.
- The usual compromise is to start with teacher forcing and slowly mix in the model’s own predictions as it improves.

4 Attention over Encoder States

Q23. Write a short note on attention in an RNN encoder–decoder. (Not exceeding three sentences.)

Answer. Instead of using only the encoder’s last memory, the decoder compares what it is doing right now against *every* memory the encoder produced, turns those comparison scores into weights with a softmax, and builds a fresh context vector as the weighted mixture of those memories. Because the weights are worked out again for every output word, the decoder can look at whichever input words matter for the word it is writing at this moment. Those weights also show, in readable form, which input word the model was looking at.

Q24. What problem does attention solve, and what does it *not* solve?

Answer.

Solves:

- The bottleneck. The input no longer has to be squeezed into one vector.
- The length penalty. Quality stops collapsing on long sentences.
- Reordering. The decoder can jump to a far-away input word when the target language needs it there.
- Opacity. You can read off which input word the model used for each output word.

Does not solve:

- The encoder is still an RNN, so training is still one word at a time and still cannot be done in parallel.
- The signal inside the encoder still has to travel step by step.
- It adds work proportional to (input length $\times$ output length) for all those comparisons.
- This leftover speed problem is exactly what “Attention Is All You Need” set out to fix.

Q25. Scenario. A legal translation system built on an RNN encoder–decoder *without* attention translates short clauses well but collapses on 60-word sentences. What would have happened if

attention had been included from the start?

Answer.

- The drop on long sentences would have been much smaller, because a fresh context vector is built for every output word instead of relying on one squeezed summary.
- Long-distance reordering, which legal text is full of, would be handled better, since the weights can jump to a far-away input word.
- Short clauses would be about the same, because the bottleneck was never the limiting factor there.
- Training would cost a little more per step, and memory would grow, because all the encoder memories have to be kept instead of just the last one.

5 Self-Attention and Scaled Dot-Product Attention

Q26. Write a short note on the query, key and value idea in self-attention. (Not exceeding three sentences.)

Answer. Each word is passed through three separate learned projections to give a query, a key and a value, borrowing the idea from search engines: the query is what you are looking for, the key is the label each item advertises itself with, and the value is what you actually get back. A word's query is compared against every word's key to decide how much of each word's value to take. It is called *self*-attention because the queries, keys and values all come from the same sentence.

Q27. In scaled dot-product attention, the scores are divided by $\sqrt{d_k}$. Explain why this scaling is needed.

Answer. A score is made by multiplying two lists of d_k numbers together and adding up the results, so the longer the lists, the bigger the totals get, roughly by a factor of $\sqrt{d_k}$. Softmax turns very big scores into a near all-or-nothing choice: one word gets almost all the weight and the rest get almost none, and in that state the gradient is almost zero, so the model stops learning. Dividing by $\sqrt{d_k}$ brings the scores back to a normal size, so softmax stays sensitive and training keeps working.

Q28. State the pros and cons of self-attention.

Answer.

Pros:

- Every word can look at every other word in one step, so distance does not weaken the link.
- All words are processed at the same time, so a GPU is used fully instead of waiting step by step.
- The attention weights show which words the model connected, which helps when something goes wrong.
- The model learns for itself which words are related, instead of being told that “nearby” means “related”.

Cons:

- The work grows with the *square* of the sentence length, because every pair of words is compared.
- On its own it has no idea of word order, so position information must be added separately.

- It has no built-in hint that nearby words matter, so it needs a lot of data to learn what an RNN gets for free.
- When generating text it must store the keys and values of every previous word, and that stored pile becomes the memory bottleneck.

Q29. What new problems did self-attention give rise to?

Answer.

- **Cost that grows with the square of the length.** Doubling the context makes attention four times as expensive, which is what limits long-context models today.
- **No sense of order.** Self-attention treats the sentence as a bag of words, so positional encodings had to be invented.
- **A large stored pile of keys and values** when generating. Serving long conversations became a memory problem, which is what grouped-query attention was built to fix.
- **Hunger for data.** Without any built-in assumption about which words go together, Transformers need far more text than RNNs to reach good accuracy.

Q30. When should self-attention *not* be used?

Answer.

- When sequences are very long and the budget is small, because the cost grows with the square of the length.
- When you must answer after every single arriving item using a fixed amount of memory, since attention has to look back over a growing pile.
- When training data is very small, because the model has no built-in hint about which words are likely to be related.
- When the task really only needs a small nearby window, where a convolution is cheaper and just as good.

6 Multi-Head Attention

Q31. Write a short note on multi-head attention. (Not exceeding three sentences.)

Answer. Instead of doing attention once over the full model width, the input is split into h smaller pieces, attention is done separately inside each piece, and the results are joined back together and passed through one more linear layer. In the original paper a width of 512 is split into 8 heads of size 64, so the total amount of arithmetic stays about the same as one full-width head. The gain is that each head can watch a different kind of relationship, such as one head following grammar and another following what a pronoun refers to.

Q32. Counterfactual. An engineer replaces the 8 heads of size 64 with 1 head of size 512, keeping the total number of weights the same. What would have happened?

Answer.

- Each layer would produce only one attention pattern, so it has to compromise between different relationships instead of handling them separately.
- Accuracy would drop on tasks that need several kinds of link at once, such as translation where the verb must agree with a far-away subject.
- The attention pictures would get harder to read: one blurry average instead of several clear, specialised ones.

- Speed would hardly change, because the total amount of arithmetic is about the same.
- The model would need more layers to make up the difference, since it can now only track one kind of relationship per layer.

7 Positional Encoding

Q33. Why does a Transformer need positional encoding at all? (Not exceeding three sentences.)

Answer. Self-attention builds each word’s output as a weighted mixture of all the words, and a mixture does not care what order things were added in, so “dog bites man” and “man bites dog” would come out identical. An RNN got order for free, because it physically reads words one after another, but the Transformer threw the recurrence away. So position has to be put back in by hand, by adding a position-dependent vector to each word embedding.

Q34. Write a short note on sinusoidal positional encoding. (Not exceeding three sentences.)

Answer. Each position k gets a vector the same size as the word embedding, whose entries alternate between $\sin(k/n^{2i/d})$ and $\cos(k/n^{2i/d})$, with n set to 10000 in the original paper. Some entries change quickly from one position to the next and some change very slowly, so together they record both the exact position and the rough region of the sentence. Because it is a fixed formula, position 7 looks the same in every sentence, and positions longer than anything seen in training can still be worked out.

Q35. Why use a repeating wave function, instead of simply using the position number itself?

Answer.

- A plain position number keeps growing, so position 5000 would be a huge value that drowns out the word embedding it is added to.
- A wave always stays between -1 and 1 , which is the same size range as the embedding values, so adding them is safe.
- Using several different wave speeds lets one vector carry both “exactly which position” and “roughly how far apart”.
- There is a neat mathematical bonus: the encoding for a position a fixed distance later can be worked out from the current one by a fixed rule, so the model finds relative distance easy to learn.

Q36. Counterfactual. A team removes positional encoding from a Transformer used for sentiment classification, and reports that accuracy barely drops. Explain what happened and why this result is misleading.

Answer.

- Sentiment is mostly about *which words are present*. If “terrible” or “loved it” is in the review, the label is usually decided, whatever the order.
- Without positional encoding the model is blind to order, but for this particular task order hardly matters, so the score stays high.
- The same model would fail badly on translation or question answering, where “the dog chased the cat” must not mean the same as “the cat chased the dog”.
- The lesson: a good score on one task does not prove a part is useless. It proves that task did not need what the part provides.

8 Residual Connections, Add & Norm, and the Feed-Forward Layer

Q37. An engineer removes all residual connections from a 12-layer Transformer encoder. Predict two likely training problems. (Not exceeding three sentences.)

Answer. First, the error signal now has to squeeze through every layer's transformation on the way back, so by the time it reaches the bottom layers there is very little left and those layers barely learn, which shows up as a loss that stops improving far too early. Second, each layer's output now completely replaces its input instead of just adjusting it, so the word's own identity and its position information get painted over as the stack gets deeper. In practice training becomes unstable and very fussy about learning rate and starting values.

Q38. Write a short note on the Add & Norm layer. (Not exceeding three sentences.)

Answer. "Add" is the residual connection: the layer's input is added back onto its output, so the layer only has to learn a small correction instead of rebuilding everything from scratch. "Norm" is layer normalisation, which rescales each word's vector so its numbers have an average of zero and a standard spread of one. Together they keep information alive through a deep stack and stop the numbers from drifting larger or smaller as they go up.

Q39. Why is *layer* normalisation used rather than *batch* normalisation? (Not exceeding three sentences.)

Answer. Batch normalisation works out its average across all the sentences processed together, so a word's numbers end up depending on which other sentences happened to be beside it, and you also have to keep running averages to use later. In NLP the sentences in a batch have different lengths, so the batch is full of padding, and that padding ruins the average. Layer normalisation uses only the numbers inside one word's own vector, so the batch and the padding make no difference, and it behaves exactly the same during training and in real use.

Q40. What problem does the position-wise feed-forward sublayer solve?

Answer.

- Attention only produces a weighted *mixture* of value vectors, and mixing is a linear operation, so stacking mixers on top of mixers adds very little real power.
- The feed-forward layer adds a non-linear step, so the model can actually reshape the mixed information instead of just re-mixing it.
- It widens the vector (from 512 to 2048 in the original paper) and narrows it again, giving room for richer processing of each word on its own.
- It is applied to every word separately and identically, so it adds no extra cost across positions and stays fully parallel.

9 The Transformer Decoder: Masking and Cross-Attention

Q41. Why is masking required in decoder self-attention but not in the encoder? (Not exceeding three sentences.)

Answer. The decoder's job is to predict word $t + 1$ from words 1 to t , and during training the whole correct answer is fed in at once for speed, so without a mask position t could just read the answer sitting at position $t + 1$. The mask sets those future scores to minus infinity before the softmax, which makes their weights zero and forces the model to use only the past. The

encoder does not need this, because it is not predicting its own input; it is describing an input sentence that is fully available anyway.

Q42. Write a short note on encoder–decoder (cross) attention. (Not exceeding three sentences.)

Answer. In cross-attention the queries come from the decoder’s current state, while the keys and values come from the encoder’s output. This lets the decoder ask “which input words matter for the word I am writing now?” and pull that information across. It is the part that actually carries the meaning of the input into the output, and it is the direct descendant of attention in the RNN encoder–decoder.

Q43. What are the three attention blocks in a full Transformer, and how do they differ?

Answer.

- **Encoder self-attention.** Q, K and V all come from the input. No mask, so every input word sees every other input word, in both directions.
- **Decoder masked self-attention.** Q, K and V all come from the output written so far. Masked, so each word sees only itself and the words before it.
- **Encoder–decoder cross-attention.** Q comes from the decoder, K and V come from the encoder. No mask over the input, because the whole input is allowed to be seen.

Q44. Counterfactual. A student trains a Transformer decoder and forgets to apply the causal mask. Training loss drops almost to zero within one epoch, but generated text is gibberish. Explain what happened.

Answer.

- With no mask, position t can look at position $t + 1$, which is literally holding the word it is being asked to guess.
- So the model learns to copy the next word rather than learning language. That is why the loss falls so fast.
- When the model is actually used, the future has not been written yet, the shortcut is gone, and the model has learned nothing useful to fall back on.
- The giveaway is exactly this pattern: near-zero training loss with useless output. Suspect a leak, not undertraining.
- The fix is to add the mask before the softmax and train again from scratch.

Q45. Scenario. You must build a system that reads a long English contract and writes a short Bangla summary of it. Which architecture should be used, and why?

Answer. An encoder–decoder Transformer fits best. The contract is fully available before you start, so an unmasked encoder can read it in both directions and build the best possible description of it; the summary is written left to right, which the masked decoder handles; and cross-attention lets each Bangla word pull from whichever part of the contract it needs. A decoder-only model can also do this by pasting the contract into the prompt, but it must read the contract left to right only and keeps it inside the same growing context, which is wasteful when the input is long and the output is short.

10 Tokenization: Byte-Pair Encoding and WordPiece

Q46. Write a short note on byte-pair encoding. (Not exceeding three sentences.)

Answer. BPE starts with single characters and keeps merging the most common neighbouring pair into a new symbol until the vocabulary reaches the size you asked for; the standard illustration is `aaabdaaabc` becoming `ZabdZabac` with $Z = aa$, then `ZYdZYac` with $Y = ab$, then `XdXac` with $X = ZY$. Common whole words end up as one token, while rare words get split into familiar pieces. This keeps the vocabulary small while still being able to write down any word at all.

Q47. What problem does subword tokenization solve?

Answer.

- **Unknown words.** A word the model has never seen can still be spelled out using pieces it knows, so nothing has to be thrown away as “unknown”.
- **Vocabulary size.** Listing every whole word in a language with lots of word endings runs into millions of entries; subwords keep it to tens of thousands.
- **Wasted learning.** “low”, “lower” and “lowest” share a stem, and subwords let the model reuse what it learned about that stem.
- **Compound words.** German “Krankenhaus” splits into pieces meaning sick and house, which a whole-word list would have to memorise separately.
- It does all this without throwing away the endings, which is what stemming and lemmatization do.

Q48. What new problems did subword tokenization give rise to?

Answer.

- Sentences become longer when counted in tokens, so attention costs more for the same amount of text.
- Splits can be linguistically wrong, because merges are chosen by how common a pair is, not by what it means.
- Languages that appear less often in the training text get chopped into many more pieces, so the same sentence costs more tokens and more money.
- Letter-level tasks get oddly hard: a model that sees “straw”+“berry” finds counting the letter r surprisingly difficult.
- The tokenizer becomes a fixed part of the model. Change it and every pretrained embedding is worthless.

11 BERT

Q49. Write a short note on BERT. (Not exceeding three sentences.)

Answer. BERT is a stack of Transformer *encoders* trained on plain unlabelled text so that every word is described using the words on both sides of it, in every layer. It is pretrained on two tasks that need no human labels, masked language modelling and next sentence prediction, and is then adapted to a real job by adding a single output layer on top and fine-tuning. The base model has 12 encoder layers, 12 heads and 768 hidden units (110 million weights); the large model has 24 layers, 16 heads and 1024 hidden units (340 million).

Q50. Describe masked language modelling, including the 80/10/10 rule. (Not exceeding three sentences.)

Answer. BERT picks 15% of the word positions at random to be predicted, and of those, 80% are swapped for the [MASK] token, 10% are swapped for a random word, and 10% are left exactly

as they were. The model must then name the original word at every picked position. Because it cannot tell which positions were tampered with, it is forced to build a good description of every word, not just the obviously blanked ones.

Q51. Why are 10% of the selected tokens replaced with a *random* token and 10% left *unchanged*, instead of masking all 15%?

Answer.

- [MASK] never shows up when the model is actually used on a real task, so training only on masked inputs leaves a mismatch between training and use.
- The 10% left unchanged force the model to keep describing words properly even when it can see them, since it does not know which ones are being masked.
- The 10% replaced at random stop the model from blindly trusting whatever word is sitting there. It has to check that the word actually fits the sentence.
- Together these make the description useful at *every* position, and that is what the fine-tuned task actually reads.

Q52. What problem does next sentence prediction solve? (Not exceeding three sentences.)

Answer. Masked language modelling teaches the model about words inside one sentence, but tells it nothing about how two sentences relate to each other. Next sentence prediction shows it pairs where half the time B really does follow A and half the time B is a random sentence from somewhere else, so it has to learn what makes two sentences belong together. That is what makes BERT directly useful for question answering and inference, which are fundamentally about the relationship between two pieces of text.

Q53. State the pros and cons of BERT.

Answer.

Pros:

- Every word is understood using the words before it *and* after it, at every layer.
- One pretrained model can be adapted to many jobs by adding just one output layer.
- Fine-tuning needs relatively little labelled data, because the language knowledge is already there.
- It beat the previous best on nearly every understanding benchmark when it came out.

Cons:

- It cannot write text properly, because it was never trained to predict the next word.
- Pretraining is expensive and wasteful: only about 15% of positions produce a learning signal each time, so it needs a lot of passes over the data.
- Training and real use do not match, because [MASK] appears in one and never in the other.
- The input is capped at 512 tokens, so long documents must be cut into chunks.
- Its knowledge is frozen at pretraining time, and it has no way to look anything up.

Q54. When should BERT be used, and when should it not be used?

Answer.

Use it when:

- The job is sorting, labelling or pulling information out: sentiment, names, finding the answer span in a passage.

- You have a few thousand labelled examples and want high accuracy cheaply.
- Speed and cost matter, since a 110-million-weight encoder is far cheaper to run than a model with billions.
- The whole input arrives at once and fits within 512 tokens.

Do not use it when:

- You need the model to write: summaries, translations, chat replies.
- You have no labelled data at all and want to work purely from instructions in a prompt.
- Documents are long and cutting them into chunks would break the reasoning.

Q55. Scenario. A bank wants to pull the borrower name and loan amount out of scanned loan applications. They have 4,000 labelled documents and a strict 50 ms time limit. Which model should they use, and why?

Answer. Fine-tune a BERT-base encoder to label each token. The job is pulling out text that is already on the page, not writing new text, so a two-directional encoder is exactly right; 4,000 labelled documents is plenty to fine-tune a pretrained encoder but far too few to train anything from scratch; and a 110-million-weight encoder finishes one forward pass well inside 50 ms. A large generative model would need one slow step per output word and could also invent a borrower name that is not on the page at all.

12 The GPT Family

Q56. Write a short note on the GPT architecture and its training objective. (Not exceeding three sentences.)

Answer. GPT is built only from the *decoder* half of the Transformer, so it uses masked self-attention and has no encoder and no cross-attention. It is trained on one job: predict the next token given all the previous ones, scored with cross-entropy against the true next token, using teacher forcing while training and feeding its own output back in when generating. That single simple job turns out to teach it far more than plain language modelling.

Q57. What was the key conceptual shift from GPT-1 to GPT-2? (Not exceeding three sentences.)

Answer. GPT-1 used the pretrain-then-fine-tune recipe, which needed a separate labelled dataset and a separate fine-tuned copy for every task. GPT-2 argued that a big enough model trained on varied enough Internet text has already seen examples of many tasks inside that text, so you can just describe the task in the prompt and skip fine-tuning entirely, which is why the paper is called “Language Models are Unsupervised Multitask Learners”. It was a straight scale-up, from about 120 million weights to 1.5 billion, trained on WebText, built by scraping pages linked from Reddit posts with at least 3 karma.

Q58. What was the key shift from GPT-2 to GPT-3? (Not exceeding three sentences.)

Answer. GPT-3 kept the same design and scaled it to 175 billion weights trained on about 300 billion tokens from CommonCrawl, WebText, Wikipedia and two book collections. The interesting result was that learning from examples in the prompt suddenly became strong: a few examples were often enough to match a fine-tuned system. It was released only through an API, and its successor GPT-3.5 together with InstructGPT is what set off the current LLM boom.

Q59. State the pros and cons of decoder-only models compared with encoder–decoder models.

Answer.

Pros:

- One simple design and one simple training job, which makes scaling to huge sizes straightforward.
- Any task can be written as text in, text out, so one model handles many jobs with no retraining.
- Every token in the training text gives a learning signal, unlike BERT where only about 15% of positions do.
- When generating, the keys and values of earlier words can be reused instead of recomputed, which keeps it efficient.

Cons:

- The input can only be read left to right, so an early word can never be informed by a later one.
- Input and output share one growing context, so a long document competes for room with the answer.
- Generating is one forward pass per word, which is slow for long answers.
- A raw pretrained model does not follow instructions and has to be instruction-tuned and aligned before it is usable.

Q60. What problems did large decoder-only models give rise to?

Answer.

- **Hallucination.** The training job rewards answers that *sound* right, not answers that *are* right.
- **Stale knowledge.** Everything is baked in at pretraining time, and retraining to update it is enormously expensive.
- **Cost and access.** Training runs costing millions of dollars put this capability in the hands of a few organisations, and GPT-2 was the last openly released model of the family.
- **A need for alignment.** A raw next-word predictor is not helpful, honest or harmless by default, which is where the whole instruction-tuning and RLHF pipeline came from.
- **Hard evaluation.** Open-ended answers have no single right answer, and benchmark contamination became a serious worry.

13 Pretraining, Data and Scaling Laws

Q61. State the central finding of Hoffmann et al. (Chinchilla). (Not exceeding three sentences.)

Answer. For a fixed compute budget, model size and training-data size should grow together at roughly the same rate, whereas the models of the time were far too big for the amount of text they were trained on. Doubling the training data gives as much or more benefit than doubling the number of weights, so an oversized model trained on too little text underperforms a smaller one trained on more. Chinchilla proved it directly: 70 billion weights beating the 280-billion-weight Gopher.

Q62. What problem do scaling laws solve?

Answer.

- They turn “how big should my model be?” from a guess into a calculation.

- They let you predict the final loss *before* spending the money, so a doomed training run can be cancelled on paper.
- They show where the real shortage is, which for most models before 2022 was data, not weights.
- They make the smaller model the better deal, because it is also cheaper to run for the rest of its life.

Q63. What problems can scaling laws *not* solve?

Answer.

- They predict loss, not usefulness. A lower loss does not promise the model is helpful, honest or safe.
- They assume enough clean text exists, and say nothing about what to do when good text runs out.
- They ignore data quality completely, counting a token of spam the same as a token of Wikipedia.
- They only balance *training* cost, so they may recommend a model that is more expensive to run than you want.
- They cannot tell you when a brand-new ability will suddenly appear.

Q64. List the main sources of LLM pretraining data and the main risks attached to them.

Answer.

Sources:

- Common Crawl: web pages scraped at petabyte scale.
- Books, including Project Gutenberg and less legitimate libraries.
- Wikipedia: clean and well organised.
- Academic papers from arXiv and PubMed.
- Code from GitHub.

Risks:

- **Quality.** Scraped web text is full of spam, machine translation and boilerplate. Data quality matters more than raw quantity.
- **Diversity.** English and certain viewpoints are heavily over-represented, which skews what the model knows.
- **Contamination.** Benchmark test sets are sitting on the web, so the model may have already memorised the exam.
- **Licensing.** Books and code come with copyright and licence conditions attached.

14 LLaMA

Q65. Name the four architectural ingredients that LLaMA introduced relative to the original Transformer decoder.

Answer. RMSNorm in place of LayerNorm, applied before each sublayer rather than after; rotary positional embeddings; the SwiGLU activation in the feed-forward layer; and grouped multi-query attention. One correction worth making in the exam: grouped-query attention arrived in LLaMA 2, and only in the 34B and 70B versions, not in LLaMA 1.

Q66. Why is normalisation needed in a deep neural network at all? (Not exceeding three sentences.)

Answer. Without it, the size of the numbers drifts as they pass up through many layers, so a layer can end up receiving values far bigger or far smaller than it was tuned for, and the error signal on the way back drifts in the same way. Normalisation resets each word's numbers to a standard size at every layer, which keeps the error signal in a usable range and makes training far less fussy about learning rate and starting values. It is what lets you stack dozens of Transformer layers and still have them train.

Q67. Write a short note on rotary positional embeddings. (Not exceeding three sentences.)

Answer. Rather than adding a position vector to the word, RoPE spins the query and key vectors by an angle that depends on the word's position, so word 1 gets a small turn and word 50 a much bigger one. When two spun vectors are compared, the result depends only on the *difference* between their two angles, so attention naturally measures how far apart two words are rather than where each one sits. It is applied inside every attention layer instead of once at the input, and it behaves better on sentences longer than anything seen in training.

Q68. Write a short note on grouped multi-query attention. (Not exceeding three sentences.)

Answer. Normal multi-head attention gives every query head its own key and value heads; grouped-query attention keeps all the query heads but lets several of them share one set of keys and values, so the number of stored key-value sets drops sharply. This shrinks the pile of keys and values that has to be fetched from memory for every word generated, which is the real bottleneck because GPUs can do arithmetic far faster than they can move data around. It sits between full multi-head attention (best quality, biggest pile) and multi-query attention (one shared set, smallest pile, some quality lost).

Q69. Counterfactual. A team replaces grouped-query attention with full multi-head attention in a 70B chat model, keeping everything else fixed. What would have happened?

Answer.

- Generating each word would get slower, because more key and value data has to be pulled out of memory at every step.
- The stored pile of keys and values would grow several times larger, so fewer users fit on one GPU and the longest usable conversation gets shorter.
- Quality would improve slightly, since each head now gets its own keys and values instead of sharing.
- Training speed and total model size would barely change, because the thing grouped-query attention saves is a generation-time cost, not a training one.
- Note what does *not* change either way: the number of word pairs compared is the same, so the cost that grows with the square of the sentence length is untouched. Grouped-query attention buys memory and speed of data movement, not less arithmetic.

15 Long Context and Mixture of Experts

Q70. What is a context window, and why does a longer one matter? (Not exceeding three sentences.)

Answer. The context window is how much text, counted in tokens, the model can hold in mind at one time, and it has grown from about 4K in GPT-3 to 128K in GPT-4 Turbo and past a million in Gemini 1.5. A longer window lets the model work on a whole book, a whole

codebase or a full legal contract in one go, instead of in disconnected chunks. It also removes the need to build a search system for inputs that now simply fit.

Q71. What are the limitations of long-context models?

Answer.

- **Lost in the middle.** Models use what is at the start and the end of the context well, but often miss what is buried in the middle.
- **Cost grows with the square of the length.** Going from 4K to 128K makes attention about 1024 times more expensive, not 32 times.
- **Memory grows too.** The stored keys and values grow with the length, and memory often runs out before the arithmetic does.
- **The advertised length is not the usable length.** A model can accept a million tokens and still reason badly over them.
- You also pay for every token you paste in, so long contexts are expensive per question.

Q72. Write a short note on Mixture of Experts. (Not exceeding three sentences.)

Answer. A Mixture of Experts layer replaces the single feed-forward sublayer with many parallel expert sub-networks plus a router that sends each token to only a few of them, so the model can hold a huge number of weights while only using a small share of them per token. Mixtral 8x7B has 8 experts and 47 billion weights in total, but activates only 2 experts per token, which works out at about 13 billion active weights. Only the feed-forward layers are duplicated per expert; the embeddings and the attention layers are shared.

Q73. State the pros and cons of Mixture of Experts.

Answer.

Pros:

- You get the knowledge of a very large model while paying the running cost of a much smaller one.
- Different experts can specialise, say one on code and another on prose, so each token gets handled by a part suited to it.
- The amount of work grows far more slowly than the total number of weights.

Cons:

- *All* the experts must sit in memory even though only a few run, so you need room for the whole 47 billion, not just the 13 billion in use.
- The router can keep sending most tokens to the same few experts, leaving the rest untrained. Extra penalty terms are needed to stop that.
- Training is less stable and harder to tune than a plain model of the same quality.
- Spreading experts across several GPUs adds communication cost and engineering work.

Q74. Scenario. A startup has one GPU with 24 GB of memory and wants the strongest possible model for general chat. Should they choose a dense 13B model or an 8x7B Mixture of Experts model with 13B active weights? Justify.

Answer. Choose the dense 13B model. The Mixture of Experts model does the same *amount of work* per token as a 13B model, but all 47 billion weights have to be sitting in memory, because the router might pick any expert for the very next token, and that will not fit in 24

GB at any sensible precision. Mixture of Experts pays off when memory is plentiful and speed is the constraint; here the constraint is memory, which is exactly the thing it is expensive in.

16 Quantization

Q75. Write a short note on quantization. (Not exceeding three sentences.)

Answer. Quantization stores the model's numbers in a smaller format, such as int8 or float16 instead of float32, to cut memory use and running cost. Post-training quantization converts an already-trained model with no retraining, which is quick but can lose accuracy; quantization-aware training pretends to quantize during the forward pass while keeping the backward pass at full precision, which costs more but keeps accuracy better. Fewer bits means less memory, less energy, and faster arithmetic, especially with whole numbers.

Q76. What is an accumulation data type, and why is one needed? (Not exceeding three sentences.)

Answer. The accumulation type is the format used to hold the running total while values are being multiplied and added up, and it is deliberately wider than the storage type: int8 adds up into int32, and bfloat16 into float32. It is needed because even a single addition can overflow: $127 + 127 = 254$, which int8 cannot hold. Without a wider running total, so much precision would be lost during a long sum that the whole point of quantizing would be destroyed.

Q77. Why is float32 to float16 conversion straightforward, while float32 to int8 is not?

Answer.

- float16 stores numbers the same *way* as float32, just with fewer bits, so the conversion is a simple trim.
- int8 has only 256 possible values and no exponent, so a wide range of decimals has to be squeezed onto a small whole-number grid using a scale S and a zero-point Z , with $x = S(x_q - Z)$.
- Even float16 has traps: the tiny ϵ used in LayerNorm is around 10^{-12} , but the smallest float16 value is about 6×10^{-5} , so it collapses to zero and the result becomes NaN.
- Hardware support differs too. Some CPUs can store float16 but convert back to float32 to actually compute, so you get no speed-up at all.

Q78. Why must the value zero be representable exactly after quantization?

Answer.

- Zero turns up everywhere in a neural network: padding tokens, masked attention positions, and the output of ReLU on any negative input.
- If zero were stored as a slightly-not-zero number, those positions would start leaking small amounts of signal into the model when they are supposed to contribute nothing.
- That is exactly why the scheme carries a zero-point Z : it is the whole number that converts back to precisely 0.0.
- Without it, errors on values that are meant to be inactive would build up layer after layer.

17 PEFT, Pruning and Distillation

Q79. Write a short note on parameter-efficient fine-tuning. (Not exceeding three sentences.)

Answer. PEFT freezes almost all of a pretrained model’s weights and trains only a small set of extra ones, so the big model stays exactly as it was and adapting it costs very little. The two common methods are adapters, which slot small feed-forward layers in between the Transformer layers, and LoRA, which learns two small matrices whose product is added onto an existing weight matrix. Because the original weights never move, one base model can serve many tasks by swapping in different small add-on modules.

Q80. State the advantages of PEFT.

Answer.

- **Efficiency.** Only a tiny share of the weights are trained, so memory and computing drop sharply.
- **Faster results.** An add-on module trains in hours rather than days.
- **No catastrophic forgetting.** The base weights are frozen, so the model does not lose its general ability while learning the new task.
- **Less overfitting.** Fewer trainable weights means less room to simply memorise a small dataset.
- **Less data needed.** Good results are possible from a few thousand examples.
- **More accessible and more flexible.** Fine-tuning becomes possible on modest hardware, and many task modules can share one base model.

Q81. What problems can PEFT *not* solve?

Answer.

- It cannot pour in large amounts of genuinely new knowledge. The base model’s knowledge stays roughly where it was.
- It cannot rescue a heavily pruned model, because recovering from pruning needs a full fine-tune.
- It cannot make the model smaller or faster to run. The whole base model still has to execute.
- It cannot fix a base model that is simply too weak for the task.
- It cannot change the tokenizer, the context length, or anything else built into the architecture.

Q82. Write a short note on magnitude pruning versus structured pruning. (Not exceeding three sentences.)

Answer. Magnitude pruning deletes the individual weights closest to zero, leaving a matrix that is the same shape but full of holes. Structured pruning deletes whole units instead, such as neurons, attention heads or entire layers, so the matrices simply become smaller. Structured pruning usually gives a real speed-up on ordinary hardware, while scattered holes need special support to be faster at all; either way the model is fine-tuned afterwards to win back accuracy.

Q83. Write a short note on knowledge distillation. (Not exceeding three sentences.)

Answer. A small “student” model is trained to copy a large “teacher” model by matching the teacher’s full set of output probabilities, rather than only matching the single correct answer. Those probabilities carry extra information, because they also show how the teacher ranks the wrong answers, which is a much richer lesson than a bare label. DistilBERT is the standard example: about 40% smaller than BERT while keeping around 97% of its accuracy.

Q84. When should distillation be used rather than quantization or pruning?

Answer.

- When you need a genuinely small and fast model for one job, such as chat running on a phone, rather than a slightly cheaper version of the big model.
- When you want the freedom to give the student a different design, since it does not have to look like the teacher at all.
- When you have lots of unlabelled text, because the teacher can label all of it for you automatically.
- Choose quantization instead when you want the same model to fit in less memory with the least effort; choose pruning when you want to trim an existing model but keep its shape.

Q85. Scenario. A company must run a question-answering model on a phone with 2 GB of free memory and no internet connection. Their current model has 7 billion weights. What should they do?

Answer.

- A 7-billion-weight model at float16 needs roughly 14 GB just for the weights, so it cannot fit. Quantizing to int8 brings it to about 7 GB, which is still far too big.
- **Distil first.** Train a much smaller student, around 1 billion weights, using the 7B model as teacher on their own data. This is the step that actually changes the size class.
- **Then quantize.** Take the student down to int8 or int4, which brings roughly 1 billion weights to about 1 GB or less.
- Optionally apply structured pruning before the final fine-tune, since removing whole units gives a real speed-up on phone hardware.
- The point is to combine them, not to pick one: distillation changes how many weights there are, quantization changes how many bytes each weight costs.

18 Instruction Tuning and Alignment

Q86. Write a short note on instruction tuning. (Not exceeding three sentences.)

Answer. Instruction tuning is fine-tuning a pretrained model on a labelled set of instructions paired with their correct answers, for example instruction “What is the sentiment of this review?”, input “The movie was boring and too long”, output “Negative”. It teaches the model that a request written in ordinary language should be *carried out*, not just continued. The result is a model that behaves like an assistant rather than like a text predictor.

Q87. Name three challenges of instruction tuning.

Answer.

- **Data is costly.** The instruction and answer pairs have to be written or checked by people, which is slow and expensive, and the model picks up whatever mistakes and habits those writers had.
- **Narrow coverage.** The model follows instructions that look like the ones it was trained on, and handles genuinely unfamiliar phrasings much worse.
- **It teaches shape, not values.** It shows the model what a good answer looks like, but gives it no way to weigh being helpful against being harmless when the two pull in opposite directions.
- **Alignment tax.** Heavy instruction tuning can make the model weaker at raw problem solving, and can make it over-cautious or formulaic.

Q88. Write a short note on alignment through SFT and state its main limitation. (Not exceeding three sentences.)

Answer. Alignment means shaping a model’s behaviour to be helpful, honest and harmless, and supervised fine-tuning is the simplest way to do it: collect examples of ideal answers written by people, fine-tune on those answers with ordinary cross-entropy, and test on held-out prompts. Its limitation is that it only ever shows the model *one* acceptable answer, and never shows it that answer A is *better than* answer B. So it cannot express a trade-off, such as a case where being as helpful as possible would actually be harmful.

Q89. Describe the RLHF pipeline and state why it is used despite its cost. (Not exceeding three sentences.)

Answer. People are shown two model answers and say which one is better; a reward model is trained to predict those judgements; and the language model is then tuned to score well against that reward model, classically using PPO. It is used because comparisons capture the “this is better than that” information that plain fine-tuning cannot express, and it produces the most helpful models in practice, as in ChatGPT and Claude. The costs are the large and constant supply of human comparisons it needs, and the fact that PPO is notoriously unstable to train, which is why DPO has become the more practical alternative.

Q90. State the pros and cons of RLHF.

Answer.

Pros:

- It can learn from comparisons, so it captures “B is better than A”, which a single demonstration cannot say.
- Judging which of two answers is better is far easier and cheaper for a person than writing the perfect answer from scratch.
- It tunes the model towards the behaviour people actually want, rather than towards guessing the next word.
- It can handle trade-offs between being helpful and being harmless.

Cons:

- It needs a large and continually refreshed supply of human comparisons, which is slow and expensive.
- PPO training is unstable and very sensitive to its settings.
- The model can learn to game the reward model instead of genuinely improving, which is called reward hacking.
- People disagree with each other and bring their own biases, and the reward model bakes those in.
- Someone has to read harmful content in order to rate it.

Q91. Write a short note on Constitutional AI. (Not exceeding three sentences.)

Answer. Constitutional AI, developed by Anthropic, aligns a model to be helpful, honest and harmless using a written set of principles instead of thousands of human ratings of toxic answers. The model is red-teamed, then criticises its own answers against those written principles, rewrites them, and learns from that self-generated feedback through supervised and reinforcement learning. This cuts the need for human-written data dramatically, and spares people from having to read harmful material.

Q92. What problem does Constitutional AI solve, and what problem does it give rise to?

Answer.

Solves:

- The human cost of alignment: far fewer people are needed to label harmful content.
- The human harm of alignment: raters no longer have to read thousands of toxic answers.
- Inconsistency: a written set of rules is applied the same way every time, whereas different people judge differently.
- Openness: the rules are written down, so they can be read, questioned and argued about.

Gives rise to:

- Someone still has to write the rules, so the value judgements are concentrated in a few authors rather than removed.
- The model's self-criticism is only as good as the model itself, so a weak model cannot spot its own subtle mistakes.
- Training on its own feedback risks reinforcing the blind spots it already has.

Q93. Scenario. A team fine-tunes a base model on 50,000 high-quality instruction-answer pairs. The model now follows instructions well, but when asked how to do something dangerous it explains it clearly and politely. What went wrong, and what is the solution?

Answer.

- **What went wrong.** Fine-tuning taught the shape of a good answer, not which answers are preferred. Every training example showed a helpful, well-written reply, so the model learned that a good answer is a helpful one, with no hint that refusing is ever better.
- **Why more of the same will not fix it.** Adding refusal examples teaches refusal for those exact prompts, but gives the model no general idea that being harmless can outrank being helpful.
- **Fix 1: RLHF.** Collect pairs where one answer complies and one refuses, and have people mark the refusal as better. The reward model then learns the trade-off itself and applies it to prompts nobody wrote examples for.
- **Fix 2: Constitutional AI.** Give the model written principles and have it criticise and rewrite its own answers against them. Much cheaper, and nobody has to read the harmful outputs.
- The missing ingredient in both cases is *comparison*, not more demonstrations.

19 Prompting, In-Context Learning and Chain of Thought

Q94. Write a short note on in-context learning. (Not exceeding three sentences.)

Answer. In-context learning is picking up a task from examples placed in the prompt, with no change to the model's weights at all: one example is one-shot, several are few-shot, and few-shot usually beats giving no examples by a wide margin. Its advantages are zero training cost and instant flexibility, since changing the task just means changing the prompt. It works because a large model trained on varied Internet text has already seen many examples of tasks laid out this way, and can recognise which kind of task the prompt is showing it.

Q95. State the pros and cons of in-context learning.

Answer.

Pros:

- No training at all, so no GPUs, no training pipeline and no waiting.

- You can change the task instantly by editing the prompt.
- It works when you only have three or four examples, far too few to fine-tune on.
- One hosted model serves every task, so there is nothing extra to run.

Cons:

- The examples take up room in the context and get re-sent with every single request, so you pay for them every time.
- Only a handful of examples fit. Fine-tuning on 50,000 examples will beat a 5-example prompt.
- Results are sensitive to small things like the order and wording of the examples.
- Nothing is kept. Once the conversation ends the model has learned nothing.
- It only works reliably in very large models, so it is not an option for small ones.

Q96. List five practical prompt-design techniques.

Answer.

- **Role prompting.** “You are a professional data scientist”, which narrows the style and vocabulary the model draws on.
- **Structured prompting.** “Step 1: Analyse. Step 2: Propose. Step 3: Conclude”, which fixes the shape of the answer.
- **Delimiters.** Using ### or triple quotes to separate the instruction from the data, so the model cannot mix the two up.
- **Output formatting.** “Respond with JSON”, which makes the answer easy for a program to read.
- **Negative constraints.** “Do not include conversational filler.”
- **Escape hatch.** Explicitly allowing “I don’t know”, which cuts down on invented answers by making it acceptable not to answer.

Q97. What is chain-of-thought prompting, and why does it improve accuracy? (Not exceeding three sentences.)

Answer. Chain-of-thought prompting asks the model to write out its reasoning as well as its final answer. It helps because the model does a fixed amount of thinking per word it writes, so making it write out the middle steps gives it more thinking time, instead of demanding a whole multi-step answer in one go. It also puts each middle result into the text, so later steps can look back at earlier ones rather than having to hold everything in mind.

Q98. What are the limitations of chain-of-thought prompting?

Answer.

- The written reasoning is not guaranteed to be the reasoning the model actually used, so a convincing-looking chain can sit next to a wrong answer.
- A mistake made early on carries forward, and the model will usually defend it rather than go back, because it can only write forwards.
- It costs many more output words, so it is slower and more expensive.
- It only helps reliably in large models. In small ones it can make accuracy worse.
- It does not help on tasks that cannot be split into steps, such as simply recalling one fact.

Q99. When should in-context learning be used rather than fine-tuning?

Answer.

- When you only have a handful of examples.
- When the task keeps changing, so retraining would never keep up.
- When you need something working today and cannot run a training job.
- When one hosted model has to be shared across many different tasks.

Prefer fine-tuning when you have thousands of labelled examples, when the task is fixed and high-volume so re-sending examples every time is wasteful, or when you need a consistent output format that prompting keeps drifting away from.

20 Retrieval-Augmented Generation

Q100. State the problems with a pure LLM that RAG addresses. (Not exceeding three sentences.)

Answer. A plain LLM invents facts, its knowledge stops at the date its training data ended, it has no access to private or company documents, and retraining it to fix any of that is far too expensive. RAG's answer is not to retrain the model but to give it a search tool. It then answers using documents fetched at the moment of the question, rather than from memory alone.

Q101. Describe the two components of a RAG system. (Not exceeding three sentences.)

Answer. The retriever turns documents into vectors, stores them in a vector database such as Pinecone, Weaviate or FAISS, turns the incoming question into a vector too, and returns the top- k closest documents. The generator is an LLM that receives those documents plus the user's question plus some instructions, and writes an answer based on what it was given. A useful detail: small models around 7B work surprisingly well as generators when retrieval is good, because the hard part has been moved outside the model.

Q102. State three advantages of Retrieval-Augmented Generation.

Answer.

- **Fresh knowledge with no retraining.** Update the documents and the answers update immediately.
- **Traceability.** Every claim can be linked back to the document it came from, so a person can check it.
- **Fewer invented facts.** The model is working from text you handed it, instead of from whatever it happens to remember.
- **Private data.** Internal documents can be used without ever putting them into training data.
- **Low cost.** Adding a document is one cheap operation, not a training run.

Q103. What problems can RAG *not* solve?

Answer.

- It cannot teach the model a *style*, a *format* or a *skill*. Retrieval hands over facts, not behaviour.
- It cannot answer a question whose answer is not in the documents. Search will return the nearest thing it finds, and the model may use it anyway.
- It cannot answer questions that need the whole collection at once, such as “how many of our contracts mention X”, because only the top few chunks are fetched.
- It cannot stop invented facts entirely. The model can still misread or embroider a perfectly correct document.

- It does not remove cost or delay. Every question now involves a search step as well.

Q104. What new problems did RAG give rise to?

Answer.

- **Everything now rests on the search.** If the wrong documents come back, the system produces confident, neatly cited nonsense.
- **Chunking.** Documents have to be cut into pieces, and a bad cut can separate a claim from the condition that limits it.
- **Extra machinery.** You now maintain an embedding model, a vector database and a re-indexing job, none of which existed before.
- **Where the text lands.** Retrieved passages get pasted into the middle of the prompt, which is exactly where the “lost in the middle” weakness bites.
- **Disagreement.** If two retrieved documents contradict each other, the model has no principled way to decide which one to believe.

Q105. Compare RAG and fine-tuning on knowledge freshness, transparency, compute cost and hallucination risk.

Answer.

Criterion	RAG	Fine-tuning
Knowledge freshness	Immediate: update the documents and today’s answers change	Frozen at training time: needs a new training run to update
Transparency	High: the answer can point at the passage it came from	Low: the knowledge is spread across the weights and cannot be traced
Compute cost	Small one-off setup, but a recurring cost per question for search and a longer prompt	Large one-off training cost, then cheap per question
Hallucination risk	Lower for facts, since the model works from text you supplied; but a bad search gives a confident wrong answer	Higher for facts, since the model answers from memory; but the output format is more reliable

Q106. Scenario (classwork). You must build a model that imitates the writing style of a particular newspaper while also being factual. Which solution should you implement?

Answer.

- **Use both, and give each the job it is good at.**
- **Fine-tune for the style.** Style is a behaviour, not a fact. It is picked up from many examples of how sentences are put together, which is exactly what changing the weights captures. Retrieval cannot install a house style: pasting sample articles into the prompt costs room on every question and still imitates worse.
- **Use RAG for the facts.** Facts change daily and have to be checkable. Baking today’s news into the weights means retraining tomorrow, and a fine-tuned model will happily invent a plausible-sounding quotation.
- **The build.** Fine-tune a base model on the newspaper’s archive so it writes in house style, then wrap it in a retriever over a live, dated news index so every factual claim is grounded in a real article.
- **The line to write in the exam:** fine-tune for form, retrieve for content.

21 Reasoning Models and the Agent Loop

Q107. Why are standard LLMs described as “System 1” thinkers, and where do they fail? (Not exceeding three sentences.)

Answer. A standard LLM is fast, automatic and pattern-matching: it produces one word per forward pass, straight through, with no way to go back, revise or plan ahead. That is why they fail on multi-step maths, logic puzzles, planning and self-correction, all of which need you to try something, notice it is wrong and start again. Reasoning models tackle this by being fine-tuned to write out their working before committing to a final answer.

Q108. Write a short note on the agent loop. (Not exceeding three sentences.)

Answer. On each turn of the loop the agent gathers the information available, calls the LLM to think and pick an action, carries out that action, looks at what happened, and feeds that observation into the next turn, repeating until the job is done or a stopping rule fires. The pattern is usually summed up as agent = LLM + memory + planning + tool use. Every major AI organisation has ended up at essentially this same design.

Q109. State the pros and cons of agentic systems.

Answer.

Pros:

- The model can use tools, so it can do things it cannot do in its head, such as run code, search the web or query a database.
- It can check its own work: an action produces a result, so a wrong answer can be spotted and retried.
- Long jobs get broken into steps instead of being demanded in one shot.
- Memory lets it carry information across many steps.

Cons:

- Mistakes build up across turns, and one wrong step early can send the whole run off course.
- Cost and delay multiply, since every turn is at least one full LLM call.
- It can get stuck in a loop, so a stopping rule has to be enforced from outside.
- Letting a model take real actions makes its mistakes consequential, not just wrong.
- Debugging is hard, because the failure could be in the thinking, in the tool, or in how the result was read.

22 Evaluation of LLMs

Q110. State three challenges in evaluating LLMs. (Not exceeding three sentences.)

Answer. These models produce open-ended answers, so there is often no single correct answer to compare against and judging correctness becomes a matter of opinion. Alignment is a matter of opinion too, since how helpful a model is depends on who is asking. And benchmark contamination means a model trained on huge amounts of web text may already have seen the test questions, so a high score can measure memorisation rather than ability.

Q111. Name five common LLM benchmarks and what each measures.

Answer. MMLU measures accuracy across 57 different subjects; HumanEval measures code writing; BIG-Bench measures a wide mix of reasoning tasks; GSM8K measures grade-school

maths; MT-Bench measures multi-turn conversation and reasoning. Benchmarks are useful but not enough on their own, because they are fixed, they get contaminated over time, and they do not measure what real users actually care about.

Q112. State the advantages and disadvantages of LLM-as-a-judge.

Answer.

Advantages:

- Fast and scalable: thousands of judgements in minutes instead of weeks.
- It agrees with human preference surprisingly often when it is prompted properly.
- Flexible: you change what is being judged by rewriting the prompt.
- Consistent: unlike a panel of people, it applies the same standard every time.
- It can explain why it gave a score.

Disadvantages:

- **Verbosity bias:** longer answers get higher scores whether or not they are better.
- **Egocentric bias:** the judge prefers answers written in its own style.
- **Ordering bias:** whichever answer is shown first tends to win.
- **Cost:** at scale, judging is itself a large bill.
- **Trustworthiness:** the judge can be wrong in exactly the same ways as the model it is judging.

Q113. When is human evaluation the only real option?

Answer.

- When the quality being measured is a matter of taste: humour, empathy, tone, tact.
- When you are checking whether an automatic judge is any good; something has to be the ground truth.
- For a final safety review before release, where missing a problem is costly.
- It is still the gold standard, but it is expensive, people disagree with each other, and it is very hard to scale.

Q114. Scenario. A team reports that their new 7B model scores 89% on MMLU, beating GPT-4. What would you check before believing this, and why?

Answer.

- **Check for contamination first.** MMLU questions are on the public web, so check whether they appear in the training data. This is by far the most likely explanation for a result this far out of line.
- **Check how it was scored.** Whether examples were given in the prompt, how the prompt was worded, and whether answers were matched strictly or loosely all move MMLU scores a lot.
- **Check other benchmarks.** A genuinely strong model should also do well on GSM8K and MT-Bench. A contaminated one usually spikes on the single benchmark it memorised.
- **Run a fresh private test set** written after the model's training cut-off. That is the only really clean check.
- **The general lesson.** A benchmark score measures performance on one fixed set of questions, not ability, and the gap between the two grows as models are trained on more of the web.

Part II: Numerical and Worked Problems

23 Attention Calculations

N1. You are given the following query (Q), key (K) and value (V) matrices for a sequence of 2 tokens:

$$Q = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

Calculate the final output values.

Answer.

Step 1 — raw scores. Multiply Q by K turned on its side, $QK^\top$. The number in row i , column j is how well query i matches key j , where $q_1 = (1, 1)$, $q_2 = (1, 0)$, $k_1 = (1, 0)$, $k_2 = (0, 1)$:

$$QK^\top = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}.$$

Step 2 — divide by $\sqrt{d_k} = \sqrt{2} \approx 1.4142$ to keep the scores a sensible size:

$$\frac{QK^\top}{\sqrt{d_k}} = \begin{bmatrix} 0.7071 & 0.7071 \\ 0.7071 & 0 \end{bmatrix}.$$

Step 3 — softmax each row, so the weights in a row add up to 1. Row 1 has two equal numbers, so the weights are (0.5, 0.5). Row 2: $e^{0.7071} = 2.0281$ and $e^0 = 1$, which add to 3.0281, giving (0.6698, 0.3302).

Step 4 — use those weights to mix the value rows:

$$\begin{aligned} \text{row 1} &= 0.5(1, 2) + 0.5(3, 4) = (2.0000, 3.0000), \\ \text{row 2} &= 0.6698(1, 2) + 0.3302(3, 4) = (1.6605, 2.6605). \end{aligned}$$

$$\text{Final output} = \begin{bmatrix} 2.0000 & 3.0000 \\ 1.6605 & 2.6605 \end{bmatrix}.$$

N2. Repeat N1 with a decoder (causal) mask applied. Which rows change, and why?

Answer. The mask sets the score in row 1, column 2 to minus infinity, because token 1 is not allowed to see token 2. After the softmax that weight becomes exactly 0, so row 1's weights are (1, 0) and its output is simply $v_1 = (1, 2)$. Row 2 does not change at all, and stays at (1.6605, 2.6605), because token 2 was already allowed to see both tokens.

$$\text{Masked output} = \begin{bmatrix} 1.0000 & 2.0000 \\ 1.6605 & 2.6605 \end{bmatrix}.$$

The exam point: masking only ever takes weight away from the *earlier* rows. The last row is always identical with and without the mask.

24 RNN Parameter Counting

N3. A simple RNN uses one-hot inputs over a vocabulary of 30 words, a hidden size of 5, and 3 output classes. Give the shape of W , U and V and the total number of weights (ignore biases).

Answer. Use the rule that a $[a \times b]$ matrix times a $[b \times 1]$ vector gives a $[a \times 1]$ vector.

- $Wx^{(t)}$ has to turn a $[30 \times 1]$ one-hot word into a $[5 \times 1]$ memory, so W is $[5 \times 30] = 150$ weights.
- $Uh^{(t-1)}$ has to turn a $[5 \times 1]$ memory into another $[5 \times 1]$ memory, so U is $[5 \times 5] = 25$ weights.
- $Vh^{(t)}$ has to turn a $[5 \times 1]$ memory into a $[3 \times 1]$ output, so V is $[3 \times 5] = 15$ weights.

Total: $150 + 25 + 15 = 190$ weights.

Notice that the sentence length appears nowhere in this calculation. A 5-word and a 50-word sentence both use the same 190 weights; the long one just runs the loop more times.

25 Positional Encoding Values

N4. With $d = 4$ and $n = 10000$, compute the positional encoding vectors for positions $k = 0, 1, 2$, using $PE(k, 2i) = \sin(k/n^{2i/d})$ and $PE(k, 2i + 1) = \cos(k/n^{2i/d})$.

Answer. With $d = 4$, the index i takes the values 0 and 1, so there are two divisors: $10000^0 = 1$ and $10000^{0.5} = 100$.

k	dims 0,1 use $k/1$	dims 2,3 use $k/100$	$PE(k)$
0	$\sin 0, \cos 0$	$\sin 0, \cos 0$	(0, 1, 0, 1)
1	$\sin 1, \cos 1$	$\sin 0.01, \cos 0.01$	(0.8415, 0.5403, 0.0100, 1.0000)
2	$\sin 2, \cos 2$	$\sin 0.02, \cos 0.02$	(0.9093, -0.4161, 0.0200, 0.9998)

Look at what the design does: the first two numbers change a lot between neighbouring positions, so they pin down exactly where you are, while the last two barely move, so they record roughly which part of the sentence you are in. Everything stays between -1 and 1 , which is why it is safe to add to the word embedding.

26 Quantization Arithmetic

N5. A weight tensor has float32 values in the range $[-0.5, 1.5]$. Using $x = S(x_q - Z)$ with $x_q = \text{round}(x/S + Z)$ and an int8 target range $[-128, 127]$, find S and Z , then quantize $x = 1.0$ and convert it back.

Answer.

Step 1 — the scale S spreads the range of decimals over the 255 gaps available between -128 and 127 :

$$S = \frac{b - a}{q_{\max} - q_{\min}} = \frac{1.5 - (-0.5)}{127 - (-128)} = \frac{2}{255} = 0.007843.$$

Step 2 — the zero-point Z is chosen so that the smallest decimal lands on the smallest whole number:

$$Z = q_{\min} - \text{round}\left(\frac{a}{S}\right) = -128 - \text{round}(-63.75) = -128 + 64 = -64.$$

Step 3 — check the ends. $x = -0.5$ gives $\text{round}(-63.75 - 64) = -128$; $x = 1.5$ gives $\text{round}(191.25 - 64) = 127$; and $x = 0$ gives exactly $Z = -64$, which is the whole reason for having a zero-point.

Step 4 — quantize $x = 1.0$: $x_q = \text{round}(1.0/0.007843 - 64) = \text{round}(127.5 - 64) = \text{round}(63.5) = 64$.

Step 5 — convert back: $S(x_q - Z) = 0.007843 \times (64 + 64) = 1.00392$.

The round-trip error is about 0.0039, which is roughly half of one step S . That is exactly what you expect from rounding, and it is the worst case.

27 LoRA Parameter Count

N6. A weight matrix in a Transformer is 4096×4096 . Using LoRA with rank $r = 8$, how many weights are trained, and what fraction of a full fine-tune is that?

Answer. A full fine-tune trains all $4096 \times 4096 = 16,777,216$ weights.

LoRA instead learns two small matrices, B of shape 4096×8 and A of shape 8×4096 , and adds their product onto the frozen original. That is

$$8 \times (4096 + 4096) = 8 \times 8192 = 65,536 \text{ weights.}$$

That is $65,536/16,777,216 = 0.39\%$ of a full fine-tune, about 256 times fewer. The key point is that the original matrix never moves, so one base model can serve many tasks by swapping in a different pair of small matrices.

28 KV Cache and Grouped-Query Attention

N7. A model has 32 layers, 32 attention heads, head size 128, and holds a 4096-token context in float16. How big is the stored pile of keys and values with full multi-head attention, and with grouped-query attention using 8 key-value groups?

Answer.

For each token, in each layer, multi-head attention stores one key vector and one value vector for every one of the 32 heads:

$$2 \times 32 \times 128 = 8192 \text{ numbers.}$$

Across 32 layers and 4096 tokens, at 2 bytes per number:

$$8192 \times 32 \times 4096 \times 2 \text{ bytes} \approx 2.15 \text{ GB.}$$

With 8 key-value groups only 8 sets are stored instead of 32:

$$2 \times 8 \times 128 = 2048 \text{ numbers per token per layer} \Rightarrow \approx 0.54 \text{ GB.}$$

The saving is exactly $32/8 = 4$ times. Note what does *not* change: the number of query heads, the number of word pairs compared, and therefore the arithmetic cost. Grouped-query attention buys memory, not less computing.

29 Mixture of Experts, Scaling and Context Cost

N8. Mixtral 8x7B has 8 experts of 7B each but 47B weights in total, and uses 2 experts per token. Explain the 47B figure and work out the active weight count.

Answer. $8 \times 7 = 56$, not 47, because only the *feed-forward* layers are duplicated for each expert. The word embeddings and all the attention layers are shared, so they are counted once instead of eight times, and that is where the missing 9 billion goes.

The active count is about 13 billion: the shared embeddings and attention layers, plus the feed-forward layers of just the 2 chosen experts. So it does the work of a 13B model, about $13/47 \approx 28\%$ of its weights, but all 47B still have to sit in memory, because the router could pick any expert for the very next token.

N9. Using the Chinchilla guideline of roughly 20 training tokens per weight, how much text does a 70B model need? Using $C \approx 6ND$ for training FLOPs, check this against the lecture's figure of about 10^{24} FLOPs for LLaMA 2 70B.

Answer. Text needed: $20 \times 70 \times 10^9 = 1.4 \times 10^{12}$, that is about 1.4 trillion tokens.

FLOPs for a 70B model trained on 2 trillion tokens:

$$C \approx 6ND = 6 \times (70 \times 10^9) \times (2 \times 10^{12}) = 8.4 \times 10^{23} \text{ FLOPs,}$$

which is about 10^{24} and matches the lecture. The lecture also gives what that number means in practice: roughly 6000 GPUs running for 12 days, at about 2 million US dollars.

N10. A model's context window is extended from 4K to 128K tokens. By how much does the attention computation grow, and by how much does the stored pile of keys and values grow?

Answer. The length goes up by $128/4 = 32$ times.

- **Attention cost** grows with the square of the length, since every token is compared with every other one, so it goes up by $32^2 = 1024$ times.
- **Stored keys and values** grow in a straight line with the length, since it is one key and one value per token, so they go up by only 32 times.

That difference is why long-context work splits into two separate engineering problems: cutting the squared computing cost, and cutting the straight-line memory cost.

30 BERT Masking Arithmetic

N11. A training sequence has 200 tokens. Under BERT's masking scheme, how many positions are picked, and how are they split?

Answer. 15% of $200 = 30$ positions are picked to be predicted. Of those 30:

- 80% = 24 positions become [MASK];
- 10% = 3 positions become a random word from the vocabulary;
- 10% = 3 positions are left completely alone.

The loss is worked out over all 30 picked positions, including the 3 that were left alone. The other 170 positions give no learning signal at all, which is why BERT needs far more passes over the data than a next-word model, where every single position teaches it something.

Part III: Mock Final Papers

Each paper below follows the format of the real final: 4 questions, 15 marks each, 60 marks total, 80 minutes. Answer all questions, and be concise and direct. Try each paper closed-book before reading the answers.

31 Mock Paper A

1. Write short notes on (each not exceeding 3 sentences): [15]

- Teacher forcing
- Why positional encoding is needed in a Transformer but not in an RNN
- The 80/10/10 rule in BERT's masked language modelling

2. You are given the following query (Q), key (K) and value (V) matrices for a sequence of 2 tokens: [15]

$$Q = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Calculate the final V values. Then state how your answer would change if a decoder mask were applied.

3. Write short notes on: [15]

- The Chinchilla scaling result (max three sentences)
- Three limitations of long-context models
- Three advantages of knowledge distillation

4. Answer all three: [15]

- A student trains a Transformer decoder and forgets the causal mask. Training loss falls almost to zero in one epoch but the generated text is gibberish. Explain in no more than three sentences.
- Why is layer normalisation used in Transformers rather than batch normalisation?
- What is in-context learning? Give one advantage and one drawback.

Answers to Mock Paper A

1a. During training the decoder is fed the correct previous word instead of the word it actually predicted, so every step starts from a clean input and the whole output can be trained at once. That makes training fast and stable. The cost is exposure bias: in real use the model has to feed on its own imperfect output, which it never practised on.

1b. Self-attention builds each word's output as a weighted mixture of all the words, and a mixture does not care what order things came in, so "dog bites man" and "man bites dog" would look identical. An RNN needs no extra help because it physically reads words one after another, so order is built into how it works. The Transformer therefore has to add a position-dependent vector to each word embedding by hand.

1c. BERT picks 15% of positions to predict, then replaces 80% of them with [MASK], 10% with a random word, and leaves 10% untouched. The untouched 10% force it to describe words properly even when it can see them, and the random 10% stop it from blindly trusting whatever

word is sitting there. Together they reduce the mismatch caused by [MASK] never appearing in real use.

2. See N1 and N2 for the full working. Unmasked: $\begin{bmatrix} 2.0000 & 3.0000 \\ 1.6605 & 2.6605 \end{bmatrix}$. With a decoder mask, row 1 becomes (1.0000, 2.0000), because token 1 may only look at itself; row 2 is unchanged, since the last position could already see everything.

3a. Hoffmann et al. showed that for a fixed compute budget, model size and training data should grow together at about the same rate, and that most models of the time were far too big for the amount of text they were given. Doubling the data gives as much or more benefit than doubling the weights. Chinchilla proved it: 70 billion weights beat the 280-billion-weight Gopher.

3b. (i) Lost in the middle: models use the start and end of the context well but overlook what is buried in the middle. (ii) Cost grows with the square of the length, so 4K to 128K is about a 1024-fold increase. (iii) The stored keys and values grow with the context and usually run out of memory before the arithmetic becomes the limit.

3c. (i) It produces a genuinely smaller and faster model, suitable for a phone, unlike quantization which keeps the same shape. (ii) The teacher’s full set of probabilities teaches more than a plain label, because it also shows how the teacher ranks the wrong answers. (iii) The teacher can label unlimited unlabelled text automatically, so no human annotation is needed. DistilBERT is the standard result: about 40% smaller at around 97% of the accuracy.

4a. Without the mask, position t can look at position $t + 1$, which is holding the very word it is being asked to guess, so the model learns to copy rather than to model language. That is why the loss collapses so quickly. When generating, the future has not been written yet, the shortcut is gone, and nothing useful was learned.

4b. Batch normalisation works out its average across all the sentences processed together, so a word’s numbers depend on which other sentences happened to be beside it, and in NLP those batches are full of padding, which ruins the average. Layer normalisation uses only the numbers inside one word’s own vector, so batching and padding make no difference and it behaves the same during training and in real use.

4c. In-context learning is picking up a task from examples placed in the prompt, with no change to the model’s weights. Advantage: zero training cost and instant flexibility, since changing the task just means changing the prompt. Drawback: the examples eat up context and get re-sent and re-paid for on every request, and only a handful fit, so fine-tuning on thousands of examples will beat it.

32 Mock Paper B

1. Write short notes on (each not exceeding 3 sentences): [15]

- a. The vanishing gradient problem in backpropagation through time
- b. The difference between decoder self-attention and encoder–decoder attention
- c. Mixture of Experts

2. A weight tensor has float32 values in the range $[-0.5, 1.5]$ and is to be quantized to int8 using $x = S(x_q - Z)$ and $x_q = \text{round}(x/S + Z)$ over the range $[-128, 127]$. [15]

- a. Compute S and Z .
- b. Quantize $x = 1.0$ and convert it back. State the error.

c. Why must the value 0.0 be representable exactly?

3. Write short notes on: [15]

- a. LoRA (max three sentences)
- b. Three challenges of evaluating large language models
- c. Three disadvantages of using an LLM as a judge

4. Answer all three: [15]

- a. Why is scaling by $\sqrt{d_k}$ needed in scaled dot-product attention?
- b. A team replaces grouped-query attention with full multi-head attention in a 70B chat model. Predict two consequences.
- c. You must build a system that writes in a particular newspaper's style while staying factually current. Which approach do you use, and why?

Answers to Mock Paper B

1a. Backpropagation through time sends the error backwards through every word, and at each step back it is multiplied by a number. If those numbers are below one, multiplying many of them together shrinks the error towards zero, so by the time it reaches the first words there is nothing left to learn from and long-range links never get learned. The opposite failure, with numbers above one, is the exploding gradient, which is handled by capping the gradient size.

1b. In decoder self-attention the queries, keys and values all come from the output written so far, and a mask stops any position from seeing later ones. In encoder-decoder attention the queries come from the decoder while the keys and values come from the encoder, and no mask over the input is needed because the whole input is allowed to be seen. The first builds the output's own internal picture; the second pulls the input's meaning across.

1c. A Mixture of Experts layer replaces the single feed-forward sublayer with many expert sub-networks plus a router that switches on only a few per token, so total capacity is huge while the work per token stays small. Mixtral 8x7B has 47 billion weights in total but uses about 13 billion per token by picking 2 of 8 experts. Only the feed-forward layers are duplicated; embeddings and attention are shared.

2a. $S = (1.5 - (-0.5)) / (127 - (-128)) = 2 / 255 = 0.007843$ and $Z = -128 - \text{round}(-0.5/S) = -128 + 64 = -64$.

2b. $x_q = \text{round}(1.0 / 0.007843 - 64) = \text{round}(63.5) = 64$; converting back gives $0.007843 \times (64 + 64) = 1.00392$; the error is about 0.0039, roughly half of one step.

2c. Zero appears all over a neural network: padding tokens, masked attention positions, and the output of ReLU on negatives. If zero were stored as a slightly-not-zero number, those positions would leak small amounts of signal when they are supposed to contribute nothing, and the error would build up through the layers. The zero-point Z exists precisely so one whole number converts back to exactly 0.0.

3a. LoRA freezes the pretrained weight matrix and learns two small matrices whose product is added onto it, so only $r(d + k)$ weights are trained instead of $d \times k$. For a 4096×4096 matrix at rank 8 that is 65,536 weights, about 0.39% of a full fine-tune. Because the original never moves, one base model can serve many tasks by swapping small add-ons.

3b. (i) Open-ended answers have no single right answer, so judging correctness is partly opinion. (ii) Alignment is opinion too, since how helpful a model is depends on who is asking. (iii)

Benchmark contamination: a model trained on much of the web may already have seen the test questions, so the score measures memorisation rather than ability.

3c. (i) Verbosity bias: longer answers score higher whether or not they are better. (ii) Ordering bias: whichever answer is shown first tends to win. (iii) Egocentric bias: the judge prefers answers written in its own style. A fourth worth naming is trustworthiness: the judge can be wrong in exactly the same ways as the model it is judging.

4a. A score is made by multiplying two lists of d_k numbers and adding up the results, so the longer the lists the bigger the totals get, roughly by a factor of $\sqrt{d_k}$. Very big scores push the softmax into a near all-or-nothing state where one word takes almost all the weight, and in that state the gradient is almost zero, so training stops. Dividing by $\sqrt{d_k}$ brings the scores back to a normal size and keeps training working.

4b. (i) The stored pile of keys and values grows several times larger, so fewer users fit on one GPU and the longest usable conversation gets shorter. (ii) Generating each word gets slower, because more key and value data has to be pulled out of memory at every step, and moving data is the bottleneck rather than the arithmetic. Quality would improve slightly, and training cost would barely change.

4c. Use both, giving each the job it is good at: fine-tune the base model on the newspaper's archive so house style is learned into the weights, then wrap it in a RAG pipeline over a live news index so the facts are fetched at question time. Style is a behaviour learned from many examples and cannot be installed by search; facts change daily and cannot be baked into weights without constant retraining. The line to write is: fine-tune for form, retrieve for content.

33 Mock Paper C

1. Write short notes on (each not exceeding 3 sentences): [15]

- a. Byte-pair encoding
- b. Rotary positional embeddings
- c. Constitutional AI

2. A simple RNN uses one-hot inputs over a 30-word vocabulary, a hidden size of 5, and 3 output classes. [15]

- a. Give the shapes of W , U and V and the total number of weights, ignoring biases.
- b. Explain why U must be square.
- c. Explain why the sentence length does not appear in your answer.

3. Write short notes on: [15]

- a. Instruction tuning (max three sentences)
- b. Three limitations of chain-of-thought prompting
- c. Three problems that RAG cannot solve

4. Answer all three: [15]

- a. A team removes positional encoding from a Transformer used for sentiment classification and reports that accuracy barely drops. Explain why this result is misleading.
- b. A company must run a 7B question-answering model on a phone with 2 GB of memory and no internet. Describe the approach you would take.
- c. Why does supervised fine-tuning alone fail to make a model harmless, and what fixes it?

Answers to Mock Paper C

1a. BPE starts from single characters and keeps merging the most common neighbouring pair into a new symbol until the vocabulary reaches the size you asked for, as in `aaabdaaabac` $\rightarrow$ `ZabdZabac` $\rightarrow$ `ZYdZYac` $\rightarrow$ `XdXac`. Common words end up as one token while rare words get split into familiar pieces, so no word is ever unknown. It keeps the vocabulary small enough to be practical while still covering any input.

1b. RoPE spins the query and key vectors by an angle that depends on the word’s position, instead of adding a position vector to the embedding. Because comparing two spun vectors gives a result that depends only on the difference between their angles, attention naturally measures how far apart two words are. It is applied inside every attention layer and copes better with sentences longer than anything seen in training.

1c. Constitutional AI aligns a model using a written set of principles rather than thousands of human ratings of harmful answers. The model criticises its own answers against that written set, rewrites them, and learns from its own feedback through supervised and reinforcement learning. This cuts the need for human-labelled data sharply and spares people from reading toxic content.

2a. W is $[5 \times 30] = 150$, U is $[5 \times 5] = 25$, V is $[3 \times 5] = 15$; total 190 weights.

2b. U takes the memory in and must produce something the same size, because its output becomes the next memory and will go through U again at the following word. Any other shape breaks the loop after one step.

2c. The input size is decided by how one word is written down as numbers, here a 30-long one-hot vector, not by how many words there are. The number of words only decides how many times the loop runs. Because W , U and V are shared across every step, the weight count does not depend on sentence length at all, and that is exactly what lets a model trained on short sentences read long ones.

3a. Instruction tuning is fine-tuning on a labelled set of instructions and their correct answers, teaching the model that a request written in ordinary language should be carried out rather than just continued. A pretrained model asked a question might simply produce more questions; instruction tuning teaches it to answer. The result is a general-purpose model that does what the user actually meant.

3b. (i) The written reasoning is not guaranteed to be the reasoning the model really used, so a convincing chain can sit next to a wrong answer. (ii) An early mistake carries forward, because the model can only write forwards and will usually defend the mistake rather than go back. (iii) It costs many more output words, so it is slower and more expensive, and in small models it can make accuracy worse rather than better.

3c. (i) It cannot teach style, format or skill, because search hands over facts and not behaviour. (ii) It cannot answer questions that need the whole collection at once, such as counting how many documents mention something, since only the top few chunks are fetched. (iii) It cannot stop invented facts completely, because the model can still misread or embroider a correctly fetched document, and a bad search produces a confident, neatly cited wrong answer.

4a. Sentiment is mostly about which words are present: if the review contains “terrible” or “loved it”, the label is usually decided whatever the order, so an order-blind model still scores well. The same model would fail badly on translation or question answering, where “the dog chased the cat” must not mean the same as “the cat chased the dog”. The result shows the task did not need word order, not that positional encoding is useless.

4b. A 7B model needs roughly 14 GB at float16, and quantizing to int8 only brings it to about

7 GB, so quantization alone cannot close the gap. Distil the 7B model into a student of around 1 billion weights using their own data, which is the step that actually changes the size class, then quantize the student to int8 or int4 to reach about 1 GB or less, optionally pruning whole units before a final recovery fine-tune. The principle is that these methods stack: distillation changes how many weights there are, quantization changes how many bytes each one costs.

4c. Supervised fine-tuning teaches the shape of a good answer, not which answers are preferred: every example shows one good reply, so the model learns what helpful looks like but never learns that refusing can be *better than* complying. Adding refusal examples only teaches refusal for those exact prompts. The fix is to give it comparisons: RLHF, where people mark a refusal as better than a compliant answer so a reward model learns the trade-off, or Constitutional AI, where the model criticises its own answers against written principles at far lower human cost.